

Chapter 3

Problem Solving

Lecture 1

Problem Solving

- This chapter describes one kind of goal-based agent called problem-solving agent.
- Our discussion of problem solving begins with precise definition of problems and their solutions.
- There are majorly two types of problems.
 - The problem like calculation of square root of a number or sine of an angle. These can be solved through the use of deterministic procedure and success is guaranteed.
 - In real world, only few problems have straightforward solutions.
 - AI is concerned with the solution of these problems

Steps of problem solving

- Four steps involved in problem solving are:
 - Goal formulation: Helps to organize behavior by isolating and representing the task knowledge necessary to solve problem.
 - Problem formulation: Define the problem precisely with initial states, final state and acceptable solutions.
 - Searching: Find the most appropriate techniques or sequence among all possible techniques.
 - Execution: Once the search algorithm returns a solution to the problem, the solution is then executed by the agent.

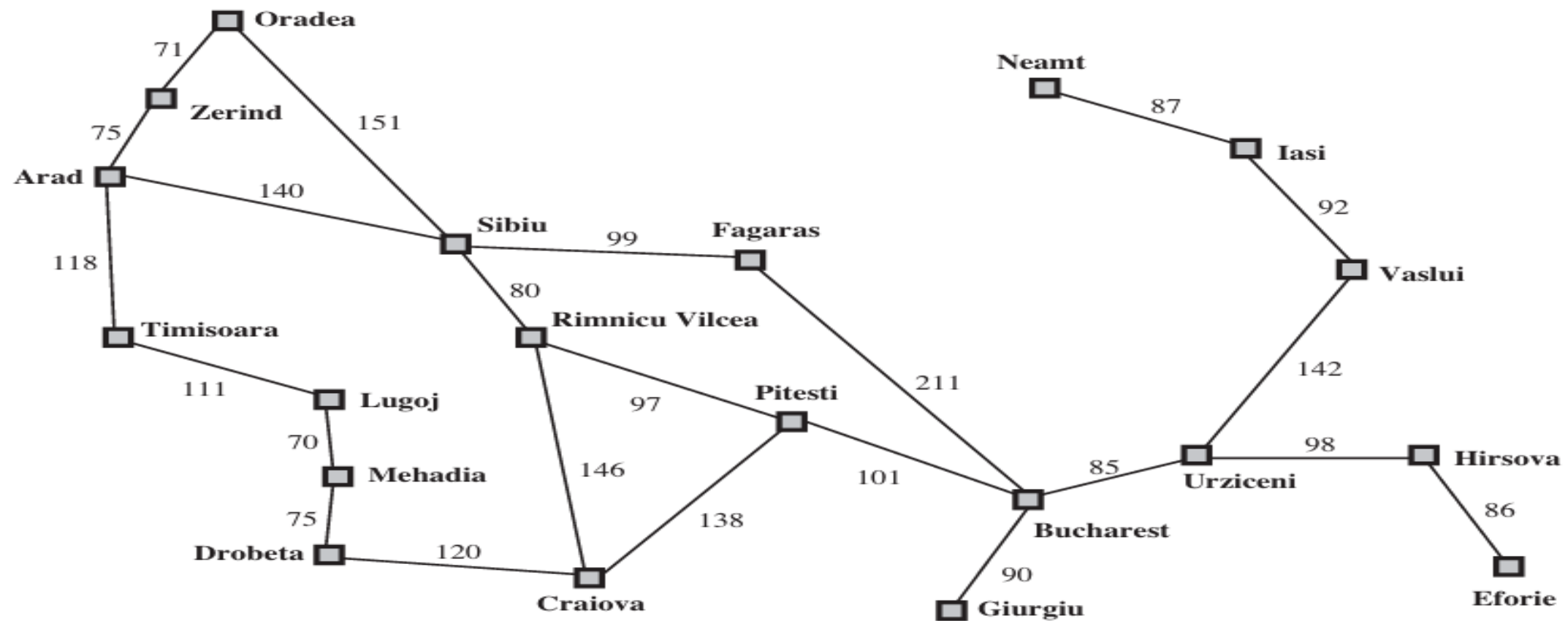


Figure 3.2 A simplified road map of part of Romania.

Well Defined Problems and Solutions

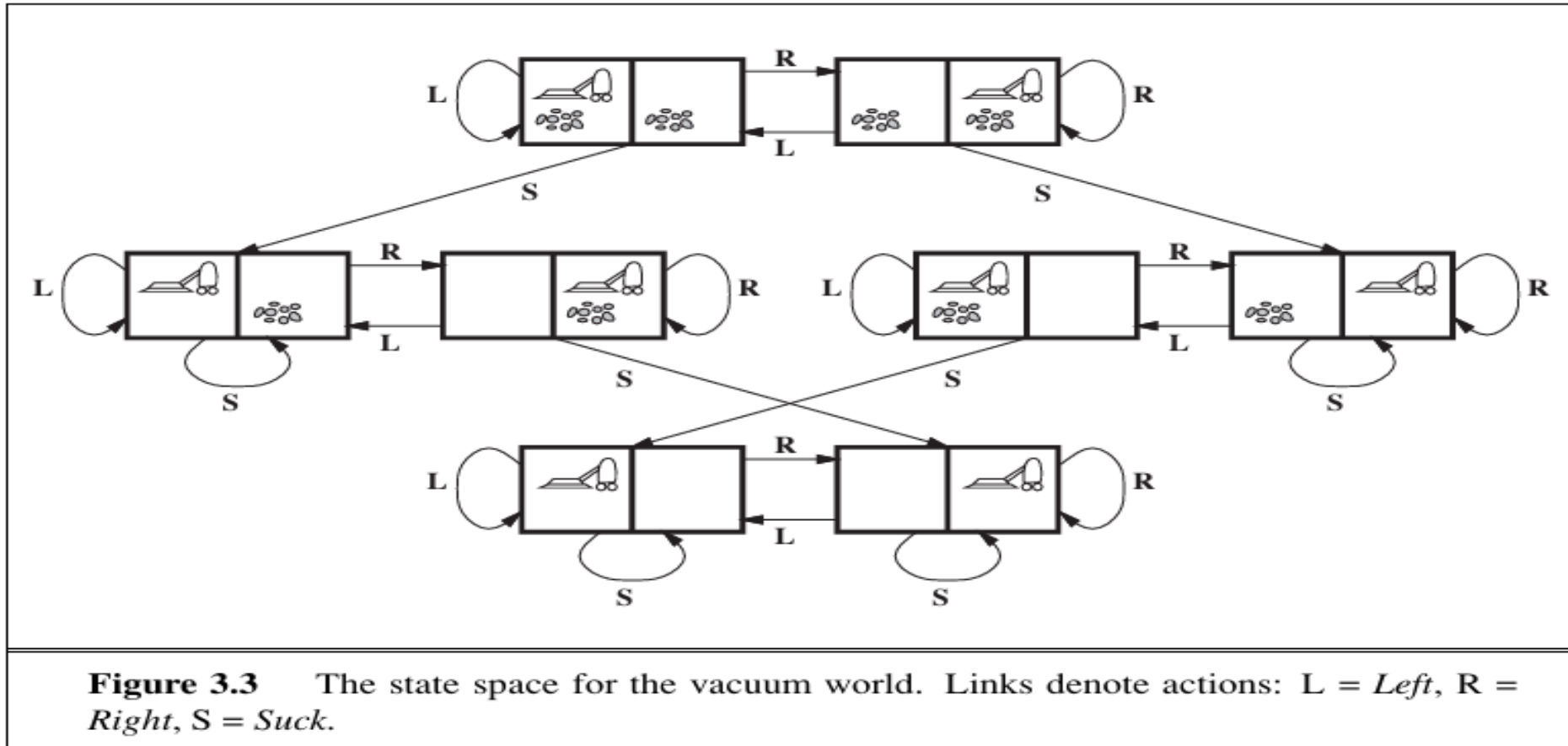
- Consider this case, an agent in the city of Arad, Romania, enjoying a touring holiday. The agent has to go from Arad to Bucharest, Romania the following day.
- Now a problem can be defined by following five components.
 - The **initial state** the agent starts in. For example, the initial state for our agent in Romania might be described as $In(Arad)$.
 - A description of the possible **actions** available to the agent. For example, from the state $In(Arad)$, the applicable actions are $\{Go(Sibiu), Go(Timisoara), Go(Zerind)\}$.
 - A description of what each action does; the formal name for this is the **transition model**, specified by a function $RESULT(s,a)$ that returns the state that results from doing action 'a' in state 's'.
 - $RESULT(In(Arad), Go(Zerind)) = In(Zerind)$.
 - The **goal test**, which determines whether a given state is a goal state. Sometimes, there is an explicit set of possible goal states, and the test simply checks whether the given state is one of them. The agent's goal in Romania is the singleton set $\{In(Bucharest)\}$.
 - A **path cost** function that assigns a numeric cost to each path. The problem-solving agent chooses a cost function that reflects its own performance measure.

Example Problems

- Toy Problems

- Let's discuss about the Vacuum Problem described in chapter 2.
- This problem can be formulated as follows:
 - **States:** The state is determined by both the agent location and the dirt locations. The agent is in one of two locations, each of which might or might not contain dirt. Thus, there are $2 \times 2^2 = 8$ possible world states. A larger environment with n locations has $n \cdot 2^n$ states.
 - **Initial State:** Any state can be considered as initial state.
 - **Actions:** In this simple environment, each state has just three actions: Left, Right, and Suck. Larger environments might also include Up and Down.
 - **Transition Model:** The actions have their expected effects, except that moving Left in the leftmost square, moving Right in the rightmost square, and Sucking in a clean square have no effect.
 - **Goal test:** This checks whether all the squares are clean.
 - **Path Cost:** Each step costs 1, so the path cost is the number of steps in the path.

Vacuum Cleaner Problem



8-puzzle Game

7	2	4
5		6
8	3	1

Start State

	1	2
3	4	5
6	7	8

Goal State

Figure 3.4 A typical instance of the 8-puzzle.

8-puzzle Game Problem formulation

- **States:** A state description specifies the location of each of the eight tiles and the blank in one of the nine squares.
- **Initial State:** Any state can be designated as the initial state.
- **Actions:** The simplest formulation defines the actions as movements of the blank space Left, Right, Up, or Down.
- **Transition Model:** Given a state and action, this returns the resulting state; for example, if we apply Left to the start state in Figure 3.4, the resulting state has the 5 and the blank switched.
- **Goal test:** This checks whether the goal state is reached or not.
- **Path cost:** Each step costs 1, so the path cost is the number of steps in the path.

Cryptarithmic

- $EGG + EGG = PAGE$
- States: A cryptarithmic puzzle with some letters replaced with digits.
- Actions: Replacing a letter with an unused digit.
- Goal test: Puzzle contains only digits.
- Path cost: Zero. All solutions equally valid.
- The solution to this puzzle is ::
- $E = ?, G = , P = ?, A = ?$.

Cryptarithmic Puzzle

- SEND+MORE=MONEY

Real world Problems

- Route finding
- Travelling Salesperson
- VLSI Layout
- Robot Navigation

Water Jug Problem

- Imagine you are given two empty jugs, one with 5 liters and another with a capacity of 3 liters. You also have access to an unlimited water source.
- Your goal is to measure exactly 4 liters of water using these two jugs.
- Constraints Involved:
 - You have a 5-liter jug and a 3-liter jug.
 - You can fill any jug completely from the water source.
 - You can pour water from one jug to another until one is empty or other is full.
 - You can empty a jug completely at any time.
 - Your objective is to get exactly 4 liters of water in one of the jugs.

Water Jug Problem

- Solution:
 - Fill the 3-liter jug completely (5 liter jug: 0L, 3-Liter Jug: 3L)
 - Pour the 3-liter jug into the 5 liter jug (3-liter jug: 0L, 5-Liter Jug: 3L)
 - Fill the 3-Liter Jug again (5-liter jug: **3 L**, 3-liter jug: **3 L**)
 - Pour from the 3-liter jug into the 5-liter jug until it is full. Since the 5-liter jug already has 3 liters, it can take only 2 more liters (5 liter jug: 5L, 3-liter jug: 1L)
 - Empty the 5-liter jug completely (5-liter jug: **0 L**, 3-liter jug: **1 L**)
 - Pour the remaining 1 liter from the 3-liter jug into the 5-liter jug (5-liter jug: **1 L**, 3-liter jug: **0 L**)
 - Fill the 3-liter jug again (5-Liter Jug: 1L, 3-liter Jug: 3L)
 - Pour all 3-liters from 3L Jug to 5L Jug (5-liter jug: **4 L**, 3-liter jug: **0 L**)
 - You now have measured 4L water successfully.

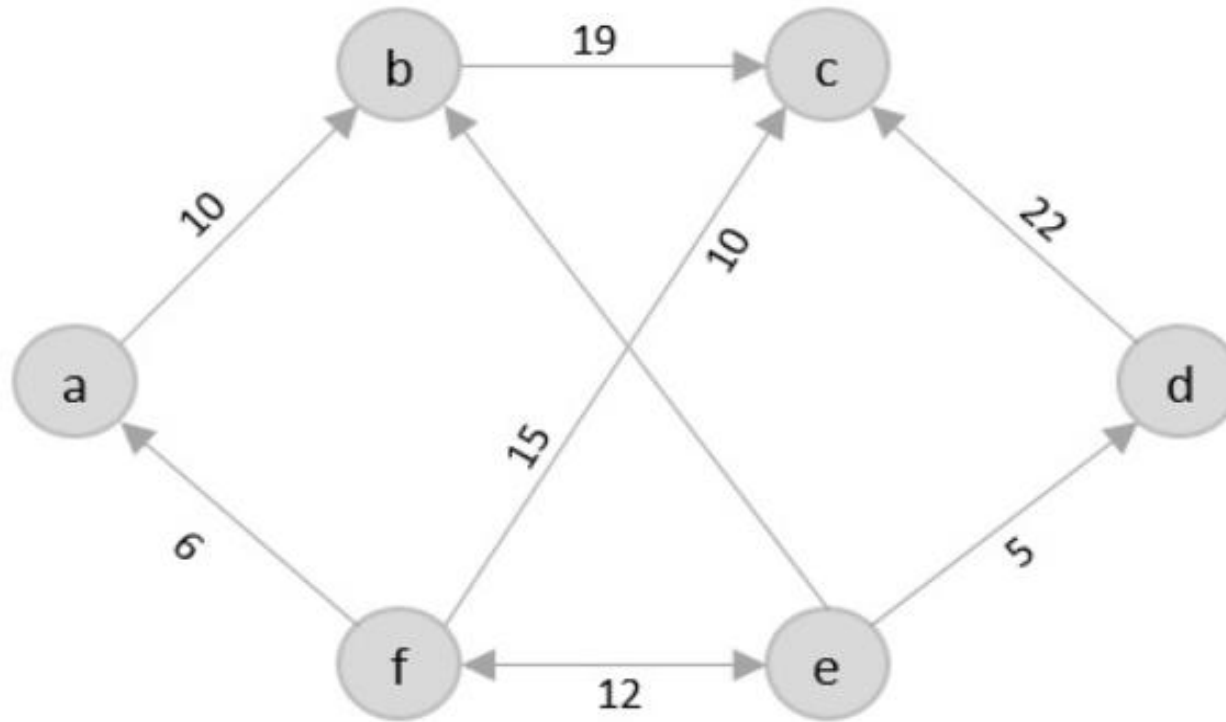
Real World Problems

- Most of the problems that we have seen till now are Toy problems and real-world problems differ from those problems in several ways.
- Touring Problems
 - Touring problems are closely related to route-finding problems but with several important differences.
 - Consider, for example, the problem “Visit every city in at least once, starting and ending in Bucharest.”
 - The state space is quite different in this case. Each state must include not just the current location but also the set of cities the agent has visited.
 - So the initial state would be $\text{In}(\text{Bucharest}), \text{Visited}(\{\text{Bucharest}\})$, a typical intermediate state would be $\text{In}(\text{Vaslui}), \text{Visited}(\{\text{Bucharest}, \text{Urziceni}, \text{Vaslui}\})$.
 - The goal test would have to check whether the agent is in Bucharest and all 20 cities have been visited.

Real World Problems

- Travelling Salesman problem
 - The travelling salesman problem is a graph computational problem where the salesman needs to visit all cities (represented using nodes in a graph) in a list just once and the distance between all of the cities are known.
 - The solution that is needed to be found for this problem is the shortest possible route in which the salesman visits all cities and returns to the origin city.
 - Let us consider a case where a salesman starts from the vertex a , they need to travel through all the remaining vertices b, c, d, e, f and get back to ' a ' while making sure that the cost taken is minimum.

Travelling Salesman problem



Travelling Salesman Problem

- If there are n cities, the number of possible tour is
 - $(n-1)! / 2$
 - This grows extremely fast.
 - For 10 cities, 181,440 possible route
 - For 20 cities, 60 *trillion* possible routes
 - It will be very difficult to brute-force.
 - TSP requires:
 - checking combinations and not just local decisions.